

pipeline stages for RISC-V subset

the work done in each step of instruction execution should take similar time to complete – since each step will be given the same time, one clock cycle

for our RISC-V subset, use following **five steps** and **corresponding pipeline stages** (a simple breakdown, but likely not ideal with respect to balancing work in each step!)

- IF: instruction fetch and update of PC
- ID: instruction decode and register reads
- EX: execution, address calculation
- MEM: memory access, branch completion
- WB: write back into register file

note that *all instructions go through the same 5 pipeline stages* but what happens in 3rd through 5th stages is dependent on the instruction type!

(based on text Fig. 4.46)

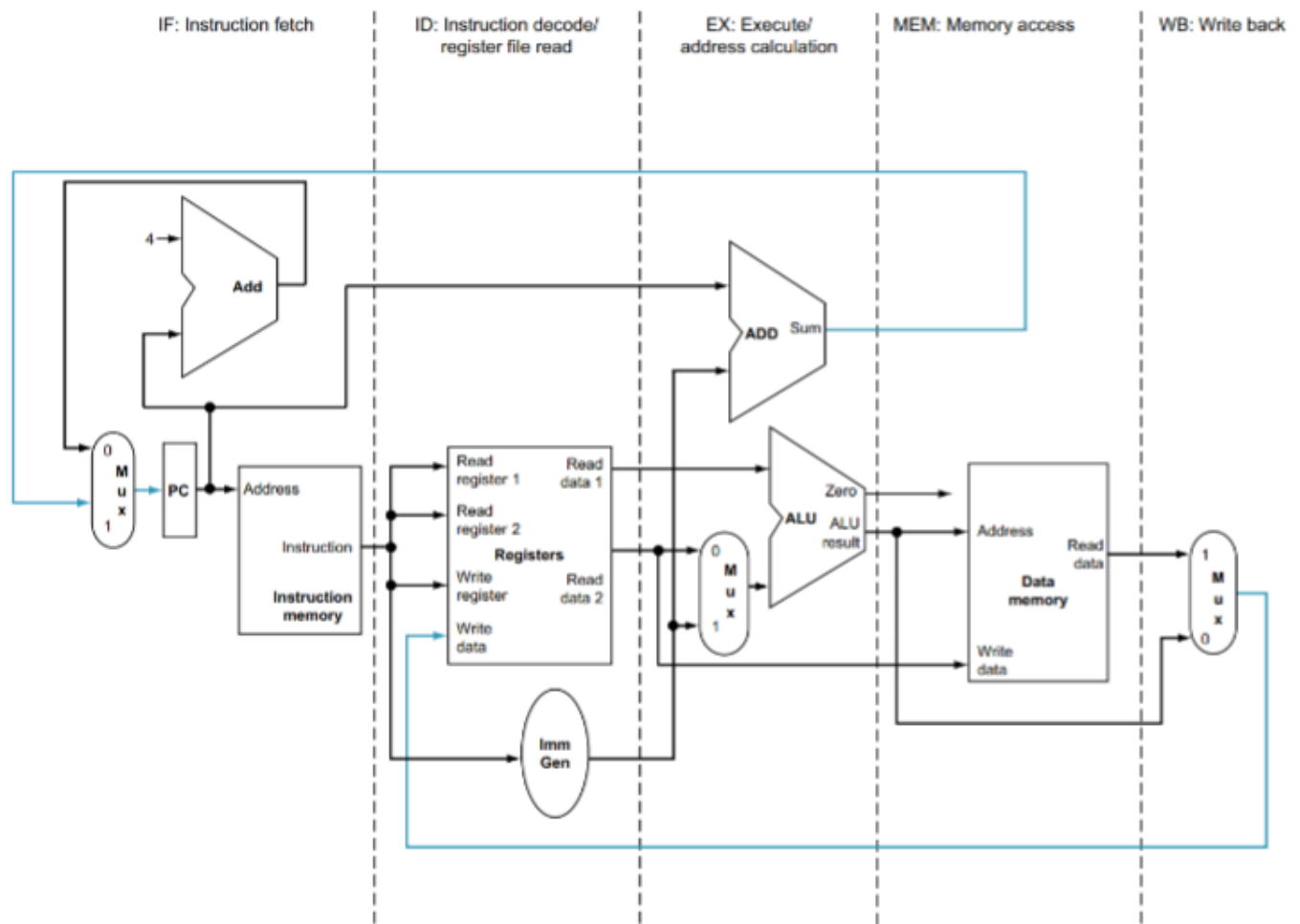
Time: CC 1 CC 2 CC 3 CC 4 CC 5 CC 6 CC 7 ...

lw x10, 40(x1)	IF	ID	EX	MEM	WB				
sub x11, x2, x3		IF	ID	EX	MEM	WB			
add x12, x3, x4			IF	ID	EX	MEM	WB		
lw x13, 48(x1)				IF	ID	EX	MEM	WB	
add x14, x5, x6					IF	ID	EX	MEM	WB

pipelined datapath design for RISC-V subset

divide our **single clock cycle datapath** into **five pieces**, corresponding to our five stages of instruction execution

(from text Fig. 4.35)

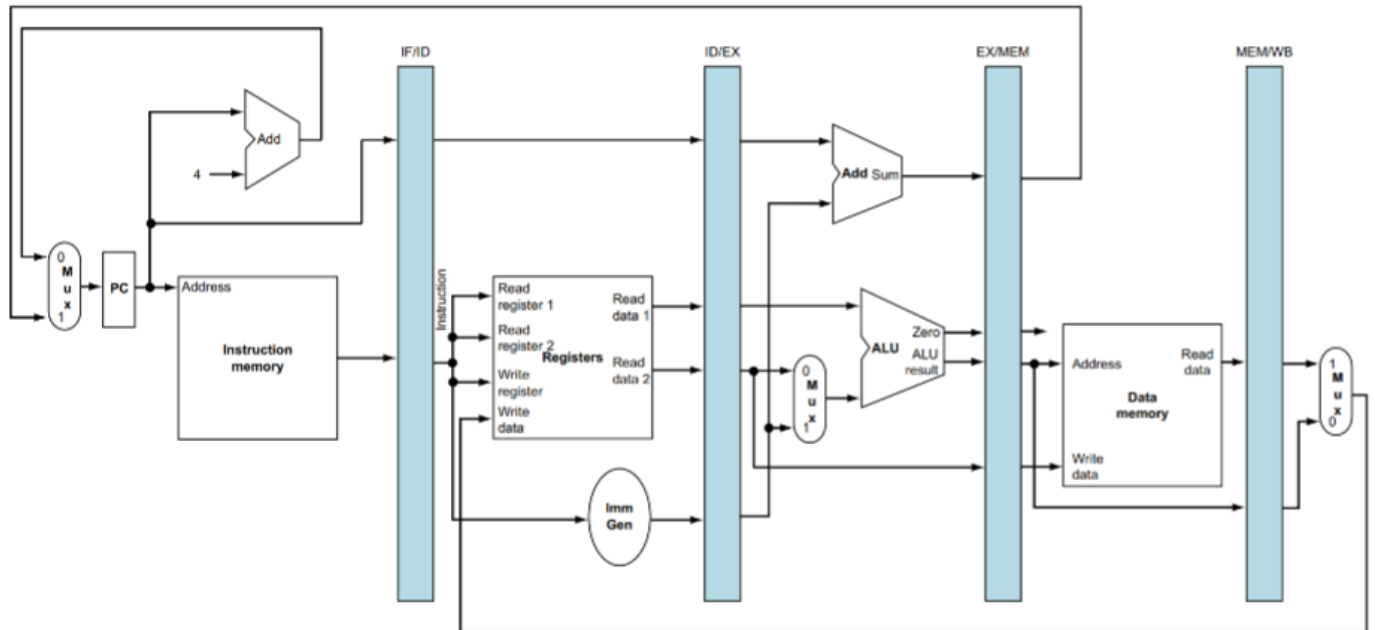


... but will need to add **pipeline registers** to hold the results of one stage of execution, for use in the next stage

- registers named **IF/ID**, **ID/EX**, **EX/MEM**, and **MEM/WB**

initial design

(from text Fig. 4.37)



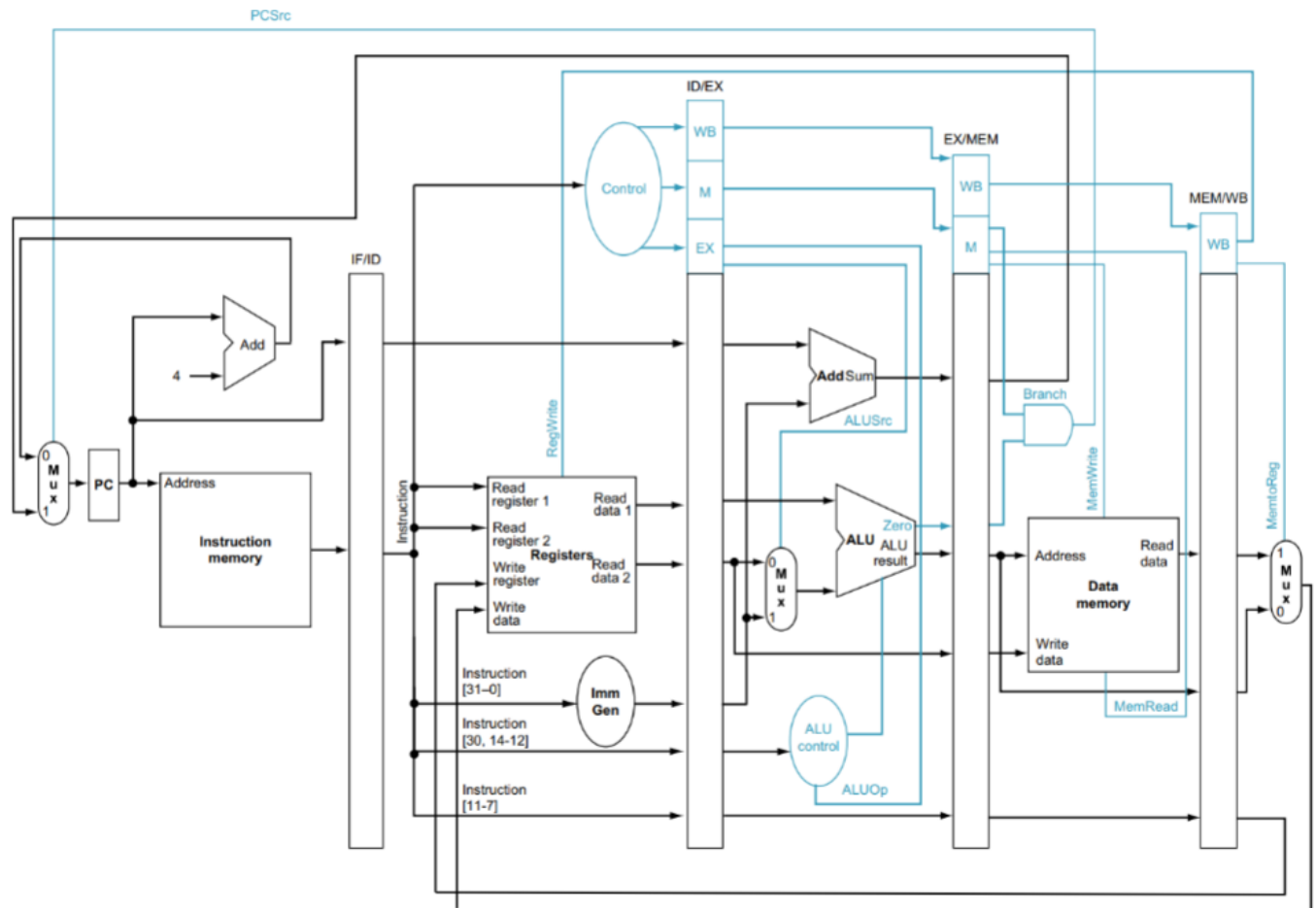
each pipeline register must be large enough to hold everything subsequent stages might need (including control signals, as we'll see shortly)

e.g., for “**sw** **s0**, **4(s1)**”, what happens in each stage?

- **IF** (first clock cycle of execution)
- **ID** (second clock cycle)
- **EX** (third clock cycle)
- **MEM** (fourth clock cycle)
- **WB** (fifth clock cycle)

how about for “**add** **s0**, **s1**, **s2**”? ... hmmm, there's a problem here, what is it?

corrected datapath, including control component
(from text Fig. 4.48)



... ignores **pipeline hazards** however!

hazards: disruptions to normal datapath operation

what types of pipelined datapath “disruptions” can you think of?

three types of hazards

- **data hazards**: can occur when an instruction needs to *read* a value *written* by a *closely preceding* instruction
- **control hazards**: occur with *conditional branches*
- **structural hazards**: occur when hardware can't support the combination of processing steps we want to perform in the same clock cycle (e.g. suppose datapath had only a single memory component used for both instructions and data)

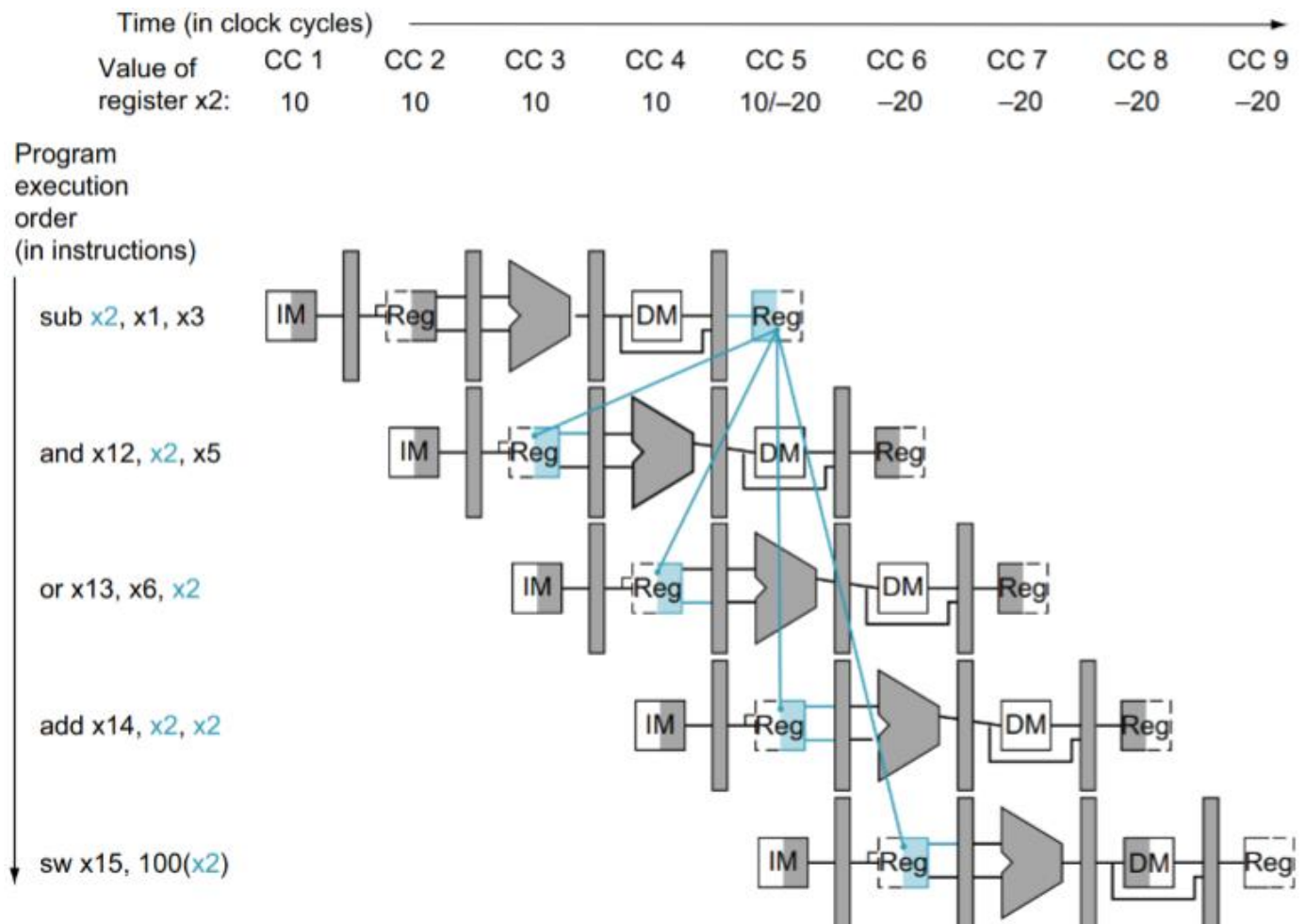
data hazards

e.g., consider following instruction sequence:

add s0, s0, t0	IF	ID	EX	M	WB			
addi s1, s0, 17		IF	ID	EX	M	WB		
lw t0, 0(s4)			IF	ID	EX	M	WB	
sw s3, 0(s4)				IF	ID	EX	M	WB

unless we somehow deal with this data hazard, we'll end up using the old contents of **s0** for the “**addi**” instruction, rather than the new value!

longer example with data hazards (from text Fig. 4.54)



colored lines show data dependencies; those going backwards in time are data hazards

how about the vertical colored line?

not a hazard ... register implementation is such that if we read a register value during the same clock cycle as that register is being written by an earlier instruction, the **new** value is read

compiler approaches for dealing with data hazards

compiler may be able to remove some data hazards through “instruction scheduling”

e.g., could remove data hazard in previous (short) example by moving the “lw” and “sw” instructions to separate the “add” and “addi”:

add s0, s0, t0	IF	ID	EX	M	WB	
lw t0, 0(s4)		IF	ID	EX	M	WB
sw s3, 0(s4)			IF	ID	EX	M WB
addi s1, s0, 17				IF	ID	EX M WB

works since the “lw” and “sw” have different operands than the “add” and “addi”, and so ordering doesn’t impact program result

but what about cases where compiler can't find instructions that can be moved in this way?

... could still remove data hazards by using “nop” instructions to separate instructions that otherwise would cause a data hazard